ASSESSMENT TOOL 1

Scenario 1: AI Drones for Emergency Medicine Delivery (Greedy Best-First vs A*)

Flood-affected region; roads become blocked suddenly; weather changes travel cost; drone has a straight-line heuristic, partial real-time blockage updates and variable movement costs.

i. Greedy Best-First Search (GBFS) in this scenario

GBFS always expands the node that *looks* closest to the goal, i.e. it uses **f(n) = h(n)** (straight-line distance to the delivery point) and ignores the cost already spent, g(n). The drone therefore flies/drives toward the hospital along whichever road reduces straight-line distance the most.

- **Strengths:** very fast decision making, small memory footprint, simple to recompute when new blockage information arrives, and it heads straight to the target, which suits an urgent mission.
- **Weaknesses under uncertainty:** it ignores terrain/weather cost, so it may pick a short-looking but costly (flooded, stormy) route; it is **not optimal** and can be **incomplete** (it may loop without a closed list); it can be trapped in dead ends or "U-shaped" blockages because a straight-line estimate does not know the road is closed; and a wrong heuristic misleads it with no correction from actual cost.

ii. A* in the same situation

A* uses **f(n) = g(n) + h(n)**. Here g(n) is the actual accumulated cost so far (distance adjusted for terrain and weather) and h(n) is the straight-line estimate of the remaining cost.

- **Balance:** g(n) pushes the search away from expensive/risky segments already travelled, while h(n) pulls it toward the goal. Nodes with a low total estimated cost are expanded first.
- **Variable costs:** when weather raises the cost of an edge, g(n) of every path through it increases, so A* automatically re-ranks routes and prefers cheaper alternatives.
- **Blocked roads:** a blocked edge is removed from the graph (cost = infinity) and the route is re-planned from the drone's current position. Incremental versions (D* Lite, LPA*) reuse earlier search effort.
- **Guarantee:** if h is admissible (never overestimates; true for straight-line distance when edge cost is at least the geometric distance) A* is complete and optimal; with a consistent h no node is re-expanded.

iii. Impact of heuristic accuracy

Heuristic quality	Greedy Best-First	A*
Accurate (close to true cost h*)	Gets near-direct path, very few expansions, but may still be non-optimal	Expands almost only nodes on the optimal path; optimal and fast
Poor / underestimating (e.g. h = 0)	Behaves randomly; wanders into wrong areas	Degrades to Dijkstra / uniform-cost search: still optimal but slow
Overestimating (inadmissible)	Not applicable as a guarantee; already non-optimal	Faster but may return a sub-optimal route, so optimality is lost
Straight-line in flood zone (misleading)	Dead ends because blocked roads are invisible to h	Still correct because g(n) captures the real cost, only more nodes are expanded

In short, GBFS depends **entirely** on heuristic accuracy, whereas A* is **robust** because the g(n) term corrects inaccuracies; heuristic accuracy for A* affects speed rather than correctness.

iv. Effect of dynamic changes (blocked paths)

- The current plan may become invalid in mid-flight, so the drone must **re-plan**; search effort already spent is partially wasted and stored g/h values become stale.
- GBFS: a newly blocked road can push it back into previously rejected branches and cause oscillation or backtracking; it has no cost memory to recover quickly.
- A*: re-running from the current node gives a correct optimal route on the updated map, but repeated full re-planning costs time. Remedies are **D* Lite / LPA*** (incremental repair), re-planning only when a blockage lies on the current path, limiting the horizon, and **weighted / anytime A*** for quick first answers that improve over time.

v. Recommended algorithm

A* with dynamic re-planning (D* Lite style) is recommended.

- **Optimality:** it finds the lowest-delay, lowest-risk route using actual terrain/weather costs, which GBFS cannot do.
- **Real-time constraints:** the straight-line heuristic keeps the search focused; weighted A* (f = g + w*h, w slightly above 1) can be used to bound computation time; incremental repair avoids planning from scratch.
- **Safety:** risk can be folded into edge cost (flood depth, wind) and blocked edges get infinite cost, so unsafe routes are avoided and the drone can retreat to a safe node when updates arrive.

Scenario 2: Smart City Traffic Signal Optimization

Optimize signal timings across hundreds of intersections to minimize total waiting time, fuel consumption and congestion; the search space is huge and traffic is dynamic.

Formulation as an optimization problem

- **State:** a complete timing plan X = (cycle length, green split and offset for every intersection).
- **Objective function:** minimize **Cost(X) = w1*TotalWaitingTime + w2*FuelConsumption + w3*CongestionIndex**, evaluated by a traffic simulator or live data.
- **Constraints:** minimum/maximum green time, pedestrian crossing time, fixed cycle bounds, emergency-vehicle priority.
- **Neighbour (move) operator:** change one green time or offset by a small step.

Why traditional search is inefficient: there is no single goal state; only the quality of the final configuration matters, not the path to it. The space is exponential (with 300 intersections and only 10 settings each there are 10^{300} plans), so BFS/DFS/A* would run out of time and memory. Each evaluation needs a simulation, traffic changes continuously so a computed optimum becomes stale, and a good admissible heuristic is hard to define. Hence **local / population-based optimization** is used.

i. Hill Climbing

Start with the current timing plan, evaluate all neighbours (small changes to one intersection), move to the neighbour with the lowest cost, and repeat until no neighbour is better. It uses constant memory and is quick and simple, so it is good for fine tuning a live system.

- **Limitations:** stops at the first local optimum, has no memory and never accepts a worse move, depends on the starting plan, and evaluates a huge neighbourhood in a city-scale problem.

ii. Local maxima, plateaus and ridges (real-world interpretation)

Issue	Meaning	Traffic interpretation
Local maximum (optimum)	Plan better than all neighbours but worse than the global best	Timing is perfect for one main road, but any single change hurts; a better city-wide plan exists but needs several roads re-timed together
Plateau	Flat area where neighbours have equal cost	During low traffic, changing a green by 1-2 seconds does not change waiting time, so the algorithm has no direction
Ridge	Narrow path to improvement not reachable by single-variable moves	A "green wave" needs offsets of consecutive junctions changed together; changing one junction alone breaks coordination and looks worse

iii. How Simulated Annealing and Genetic Algorithms overcome these limits

- **Simulated Annealing (SA):** accepts a worse move with probability $\mathbf{e^{-(delta\ E)/T}}$. At high temperature T it explores widely and can climb out of local optima and cross plateaus; as T is gradually lowered it behaves like hill climbing and settles in a good solution.
- **Genetic Algorithm (GA):** keeps a population of timing plans (chromosome = vector of green splits/offsets, fitness = 1/Cost). *Selection* keeps good plans, *crossover* combines parts of different plans (e.g. north zone of one with south zone of another) so several intersections change together (solving ridges), and *mutation* adds random variation to escape local optima. It searches many regions in parallel.

iv. Recommended approach

A **Genetic Algorithm combined with local search (memetic approach)**, with SA-style tuning for the final refinement, is recommended.

- **Scalability:** fitness evaluation of the population is easily parallelized across city zones/servers; the chromosome can be decomposed by region.
- **Adaptability:** when traffic patterns change, the existing population is reused as a warm start so the system re-optimizes quickly; it is an *anytime* algorithm that can return the best plan found so far.
- **Quality:** global exploration by GA plus fast hill-climbing refinement avoids local optima while still giving real-time responsiveness.

Scenario 3: Autonomous Mars Rover in Unknown Terrain

The rover must collect samples and avoid craters and rocks with limited sensing and no complete map; it updates knowledge while exploring.

i. Why an online search agent is needed

- **Unknown environment:** offline search needs a complete model (all states, actions and costs) before acting, but the map is unknown.
- **Interleaving:** an online agent alternates *sense - plan - act*; it only plans for what it knows and extends the plan as new areas are revealed.
- **Communication delay:** Earth-Mars signal delay (several minutes) makes remote step-by-step control impossible, so the rover decides locally.
- **Cost of mistakes:** exploring is itself the way to learn; a precomputed plan on a wrong map could drive the rover into a crater.

ii. Challenges: partial observability and dynamic environments

- **Partial observability:** sensors have limited range, noise and blind spots; hazards beyond the sensor range or hidden behind rocks are unseen; wheel slip and no GPS make the rover's own position uncertain. The agent must reason with a **belief state** rather than a certain state.
- **Dynamic changes:** dust storms reduce visibility, sand shifts, slopes change traction, lighting and battery levels vary; previously safe cells may become unsafe, so old knowledge can become outdated.
- **Other difficulties:** irreversible actions (getting stuck or falling in a crater), limited energy/time, balancing exploration vs. collecting samples, and the risk of repeatedly visiting the same places or being trapped in dead ends.

iii. How the rover builds and updates its internal model

The rover maintains a map (e.g. an **occupancy grid / SLAM map**) in which every cell stores a state such as free, obstacle, hazard or unknown, with a probability. The cycle is:

- **1. Sense:** camera, LIDAR and IMU readings give local terrain information.
- **2. Update:** cells in view are updated (Bayesian / log-odds update) and the rover's pose is estimated by matching sensor data with the map (localization).
- **3. Store:** visited cells, discovered hazards, terrain traversal cost, sample locations and energy status are saved.
- **4. Plan and act:** the best next action is computed on the current model (unknown cells treated carefully), one step is executed, and the loop repeats. When observations contradict the model, the model and plan are corrected.

iv. Online uninformed vs. informed search

Aspect	Uninformed (online DFS, BFS, uniform cost)	Informed (LRTA*, D* Lite, online A*)
Guidance	No knowledge of goal direction; explores systematically	Uses heuristic (distance to target/sample, estimated cost) to prefer promising moves
Efficiency	Many unnecessary moves, high energy use	Fewer moves, energy saving
Learning	Mostly backtracking; needs reversible actions	LRTA* updates cost estimates h(s) as it learns; D* Lite repairs the plan when the map changes
Safety	Can blindly enter risky areas	Hazard cost in heuristic helps avoid danger

Aspect	Uninformed (online DFS, BFS, uniform cost)	Informed (LRTA*, D* Lite, online A*)
Drawback	Slow, may revisit areas	Poor heuristic can mislead; needs more computation per step

v. Most suitable approach

An **informed online search agent such as D* Lite / LRTA* with a safety-aware cost function** is the best choice.

- **Uncertainty:** unknown cells are given a conservative (higher) cost and a belief/probability map is maintained, so risk is considered in planning.
- **Adaptability:** as new obstacles are sensed, D* Lite repairs the existing path incrementally instead of re-planning from scratch, saving computation and energy.
- **Safety:** hazard cells get very high/infinite cost, a safety margin is kept around craters, and a fallback behaviour (stop, retreat to the last safe cell) is available. Heuristic guidance makes exploration efficient within limited battery.

Scenario 4: University Exam Timetabling as a CSP

Schedule exams so that no student has overlapping exams, halls are limited, some subjects must precede others, faculty availability is limited and some students need special accommodation.

i. Formulation as a Constraint Satisfaction Problem (X, D, C)

Variables (X): one variable per exam (course) $\mathbf{E_c = (slot\ t_c, hall\ h_c)}$, for c = 1..n courses.

Domains (D): D_c = all pairs (time slot, hall) with the hall capacity not less than the number of students registered for course c, and slot/hall available to the faculty, e.g. slots {Day1-FN, Day1-AN, Day2-FN, ...} and halls {H1, H2, ...}.

Constraints (C):

Constraint	Type	Explanation
t(c1) != t(c2) if courses c1, c2 share a student	Binary	No student has overlapping exams
If t(c1) = t(c2) then h(c1) != h(c2)	Binary	At most one exam per hall per slot (limited halls)
capacity(h) >= enrollment(c)	Unary	Hall must be large enough (prunes the domain)
t(c1) < t(c2) for specified pairs	Binary	Certain subjects must be scheduled before others
t(c) in AvailableSlots(faculty of c)	Unary	Faculty / invigilator availability
h(c) in SpecialHalls, longer slot for c	Unary	Special accommodations (ground floor, extra time, scribe)
Max 2 exams per student per day	Soft/global	Fairness (can be treated as preference when over-constrained)

ii. Backtracking search in this scenario

Backtracking is depth-first search that assigns one variable at a time, checks consistency with already assigned variables after each assignment, and **backtracks** (undoes the last assignment and tries another value) when a constraint is violated or a domain is empty.

Mini example: courses M (Maths), P (Physics), C (Chemistry); slots {S1, S2, S3}. Constraints: M and P share students (M != P), P and C share students (P != C), M must be before C (M < C).

- Assign M = S1. Assign P = S1 violates M != P, so try P = S2 (OK).
- Assign C = S1: fails M < C. C = S2: fails P != C. C = S3: satisfies all, so the schedule M=S1, P=S2, C=S3 is a complete solution.
- If no value of C worked, the algorithm would backtrack to P (try S3) and then to M, until all constraints are satisfied or all options are exhausted.

iii. Constraint propagation (forward checking) and improved efficiency

Forward checking removes from the domains of unassigned variables every value that conflicts with the latest assignment, and backtracks immediately when any domain becomes empty. In the example: after M = S1, S1 is deleted from D(P) giving {S2, S3}, and because M < C, S1 is deleted from D(C) giving {S2, S3}. After P = S2, S2 is removed from D(C) leaving {S3}. The conflict is detected **before** it is reached, so no wasted deep search occurs.

- **Arc consistency (AC-3 / MAC)** goes further by propagating the pruning to neighbours of neighbours.
- **Heuristics:** MRV (choose the variable with the fewest legal values, e.g. courses with limited faculty availability), degree heuristic (most constrained course with many shared students) and least-constraining-value (choose the slot that rules out the fewest options for others).

- Together these reduce the number of nodes visited, find dead ends early and avoid the thrashing of plain backtracking.

iv. Real-world challenges and best strategy for scalability

- **Size:** thousands of courses/students lead to a dense conflict graph; the problem is NP-hard (similar to graph colouring), so the search tree explodes as courses and students grow.
- **Over-constrained:** a perfect solution may not exist, so some constraints must be relaxed (soft constraints with penalties).
- **Dynamic changes:** late registrations, hall or faculty changes and clashes require quick repair without rebuilding the entire timetable.
- **Fairness and quality:** spreading exams for students, hall utilization and invigilator workload.

Best strategy: a **hybrid approach**. Build the initial timetable with backtracking + forward checking/MAC and MRV + degree heuristics (hard constraints guaranteed), decompose the problem by department/semester to keep each sub-problem small, and use **min-conflicts local search** (or SA/GA for soft-constraint optimization) to repair the timetable when changes occur. This scales well and keeps the schedule adaptable.

Scenario 5: Strategic Game AI (Minimax and Alpha-Beta Pruning)

The AI must compete against humans in a real-time strategy game, decide within strict time limits and evaluate complex states with a very large decision tree.

i. Minimax modelling of the problem

The game is modelled as a two-player, zero-sum adversarial search problem. **MAX** is the AI (tries to maximize the score) and **MIN** is the human (tries to minimize it). The game tree has states as nodes, legal moves as edges and a **utility / evaluation value** at leaves.

- Minimax value: MAX nodes take the **maximum** of their children; MIN nodes take the **minimum** of their children; values are propagated bottom-up and the AI plays the move with the best backed-up value.
- It assumes the opponent plays optimally, so the chosen move gives the best guaranteed outcome in the worst case. It is executed as a depth-first recursion, so memory is only O(b*m).
- For a real-time game the continuous play is discretized into decision steps (turns / action intervals) and the tree is cut off at a limited depth, where an evaluation function replaces the true outcome.

ii. Computational challenges of large game trees

- **Exponential time:** complexity is $O(b^m)$ for branching factor b and depth m. For example with b = 30 and depth 8, about $30^8 = 6.6 \times 10^{11}$ nodes must be examined, which is impossible within a few milliseconds.
- **Large branching factor:** many units with many possible actions each make b very large.
- **Depth limit and horizon effect:** the search cannot reach terminal states, so the quality depends on the evaluation function, and important events beyond the horizon are missed.
- **Real-time limits and uncertainty:** decisions are required within strict time; hidden information (fog of war) and simultaneous actions are not captured exactly by simple minimax.

iii. How Alpha-Beta pruning improves efficiency

Alpha-Beta keeps two bounds during the search: **alpha** = best value MAX can guarantee so far on the path (lower bound), and **beta** = best value MIN can guarantee so far (upper bound). When **alpha >= beta** in a node, the remaining children cannot influence the final decision and are **pruned**.

Example: MAX root has two MIN children. Child A has leaves 3 and 5, so A = min(3,5) = 3 and alpha at the root becomes 3. In child B the first leaf is 2, so B can be at most 2 (beta = 2). Since beta (2) <= alpha (3), MAX will never choose B, so the remaining leaf of B (e.g. 9) is **not evaluated**. The final result is still 3, the same as minimax.

- **Reasoning:** pruning never changes the final minimax decision, it only removes branches that are provably irrelevant.
- **Efficiency:** with the worst ordering there is no pruning, $O(b^m)$; with the best move ordering the complexity is about $\mathbf{O(b^{m/2})}$, so the AI can search **twice as deep** in the same time. Good move ordering (best/capturing/strong moves first) is therefore crucial.

iv. Evaluation function design

A fast, weighted linear function is used: **Eval(s) = w1*f1(s) + w2*f2(s) + ... + wn*fn(s)**, where each f_i is a feature difference between AI and human (AI minus opponent) and w_i is its importance.

Feature	Reason for inclusion
f1: Army strength (number, health, type of units)	Military power decides battles and survival
f2: Resources and economy (gold, workers, income rate)	Future production and ability to sustain the army

Feature	Reason for inclusion
f3: Map / territory control (bases, key positions, vision)	Control gives mobility, safer expansion and resource access
f4: Unit position and threat (distance to enemies, formation, defence)	Good positioning wins fights and prevents surprise attacks
f5: Technology / production capacity	Long-term advantage and unit quality
f6: Base health and win/loss status	Terminal-like conditions, a very high weight for victory/defeat

The features are cheap to compute so that many nodes can be evaluated, the values are normalized to the same scale, and the weights are tuned using expert knowledge, self-play or machine learning. Terminal states (win/lose) receive extreme values.

v. Strategy for balancing optimality and real-time performance

- **Iterative deepening alpha-beta with a time limit:** search depth 1, 2, 3... and when time expires play the best move from the last completed depth (an anytime algorithm). Results of shallow searches also improve move ordering for deeper ones.
- **Move ordering and heuristics:** killer/history heuristics and trying promising moves first to maximize pruning.
- **Transposition table:** store already evaluated states so repeated positions are not searched again.
- **Selective search:** limit branching (beam / forward pruning of poor moves), quiescence search to avoid the horizon effect in unstable positions, and abstraction of units into groups to reduce b.
- **Alternatives for very large trees:** Monte Carlo Tree Search or hybrid approaches, plus precomputed opening books and time management that spends more time in critical situations.

This gives the best possible move within the available time: near-optimal decisions in simple situations and fast good-enough decisions in complex ones.

CO2 – ASSESSMENT TOOL 2 (AT2)

Artificial Intelligence (CSA17) – Annexure A: Dry Run Evaluation

Question 1 – Breadth-First Search (BFS)

Graph: A → (B, C); B → (D, E); C → (F, G). Data structure: **Queue (FIFO)**. Start node = A. Children are added left to right.

Iteration	Queue (before)	Visited Node	Queue (after expansion)
1	A	A	B, C
2	B, C	B	C, D, E
3	C, D, E	C	D, E, F, G
4	D, E, F, G	D	E, F, G
5	E, F, G	E	F, G
6	F, G	F	G
7	G	G	Empty

1. BFS traversal order: A → B → C → D → E → F → G

2. Queue contents after each iteration: [B, C] → [C, D, E] → [D, E, F, G] → [E, F, G] → [F, G] → [G] → [ ] (empty).

Question 2 – Depth-First Search (DFS)

Data structure: **Stack (LIFO)**. Start node = A. Children are pushed right to left so that the left child is on top and is explored first. Stack contents are written with the **top on the left**.

Iteration	Stack (before)	Visited Node	Stack (after expansion)
1	A	A	B, C
2	B, C	B	D, E, C
3	D, E, C	D	E, C
4	E, C	E	C
5	C	C	F, G
6	F, G	F	G
7	G	G	Empty

DFS traversal order: A → B → D → E → C → F → G

Question 3 – 8-Puzzle: Breadth-First Search

Initial state and **Goal state** (0 = blank tile):

Initial State	Goal State
1 3 6 5 0 2 4 7 8	1 2 3 4 5 6 7 8 0

Blank moves are tried in the order **Up, Down, Left, Right**. Repeated states are not added again. Each state is named S0, S1, S2 … in the order it is generated (S0 = initial state). Data structure: **Queue (FIFO)**.

BFS table – first 5 iterations

Iter.	Current State (expanded)	Queue Before Expansion	Queue After Expansion
1	S0	S0	S1, S2, S3, S4
2	S1	S1, S2, S3, S4	S2, S3, S4, S5, S6
3	S2	S2, S3, S4, S5, S6	S3, S4, S5, S6, S7, S8
4	S3	S3, S4, S5, S6, S7, S8	S4, S5, S6, S7, S8, S9, S10
5	S4	S4, S5, S6, S7, S8, S9, S10	S5, S6, S7, S8, S9, S10, S11, S12

State key (for the table above):

State	Configuration (rows separated by |)	Parent / Move
S0	1 3 6 | 5 0 2 | 4 7 8	–
S1	1 0 6 | 5 3 2 | 4 7 8	S0 / blank Up
S2	1 3 6 | 5 7 2 | 4 0 8	S0 / blank Down
S3	1 3 6 | 0 5 2 | 4 7 8	S0 / blank Left
S4	1 3 6 | 5 2 0 | 4 7 8	S0 / blank Right
S5	0 1 6 | 5 3 2 | 4 7 8	S1 / blank Left
S6	1 6 0 | 5 3 2 | 4 7 8	S1 / blank Right
S7	1 3 6 | 5 7 2 | 0 4 8	S2 / blank Left
S8	1 3 6 | 5 7 2 | 4 8 0	S2 / blank Right
S9	0 3 6 | 1 5 2 | 4 7 8	S3 / blank Up
S10	1 3 6 | 4 5 2 | 0 7 8	S3 / blank Down
S11	1 3 0 | 5 2 6 | 4 7 8	S4 / blank Up
S12	1 3 6 | 5 2 8 | 4 7 0	S4 / blank Down

Note: the goal is at depth 8, so it cannot appear in the first 5 iterations. BFS must finish the whole of depth 7 before depth 8 is reached; the goal state is the **311-th** node taken from the queue. The table above shows the first 5 iterations in full; the remaining iterations continue in the same way, level by level (summary below).

Level-wise summary of BFS (generation stops once the goal is dequeued):

Depth	States generated at this depth	Cumulative states generated
0	1	1
1	4	5
2	8	13
3	8	21
4	16	37
5	32	69
6	60	129
7	72	201
8	136	337
9	163	500

Answers to the Questions

1. Which node is expanded in each iteration?

Iteration 1 expands S0 (the initial state). Because BFS uses a FIFO queue, the node expanded in each iteration is the one at the **front** of the queue: iteration 2 → S1, iteration 3 → S2, iteration 4 → S3, iteration 5 → S4, and so on in the order S0, S1, S2, … until the goal state is taken from the queue.

2. How many states are generated in total?

500 states are generated in total (including the initial state), of which **311** are expanded before the goal is reached. The queue still holds 189 unexpanded states at that point.

3. Complete solution path from the initial state to the goal state

Moves of the blank tile (8 moves): **Right → Up → Left → Down → Left → Down → Right → Right**

Start 1 3 6 5 0 2 4 7 8	Right 1 3 6 5 2 0 4 7 8	Up 1 3 0 5 2 6 4 7 8	Left 1 0 3 5 2 6 4 7 8	Down 1 2 3 5 0 6 4 7 8
Left 1 2 3 0 5 6 4 7 8	Down 1 2 3 4 5 6 0 7 8	Right 1 2 3 4 5 6 7 0 8	Right 1 2 3 4 5 6 7 8 0	

Solution depth = 8, path cost = 8. This is the shortest possible solution.

4. Why does BFS always find the shortest solution in an unweighted 8-puzzle?

BFS expands nodes level by level: every state at depth d is expanded before any state at depth d+1. In the unweighted 8-puzzle every move costs 1, so the path cost equals the depth of the node. Therefore the first time the goal appears, it is at the smallest possible depth, and no shorter path can exist (any shorter path would have been found at an earlier level). Hence BFS is complete and optimal for uniform step cost.